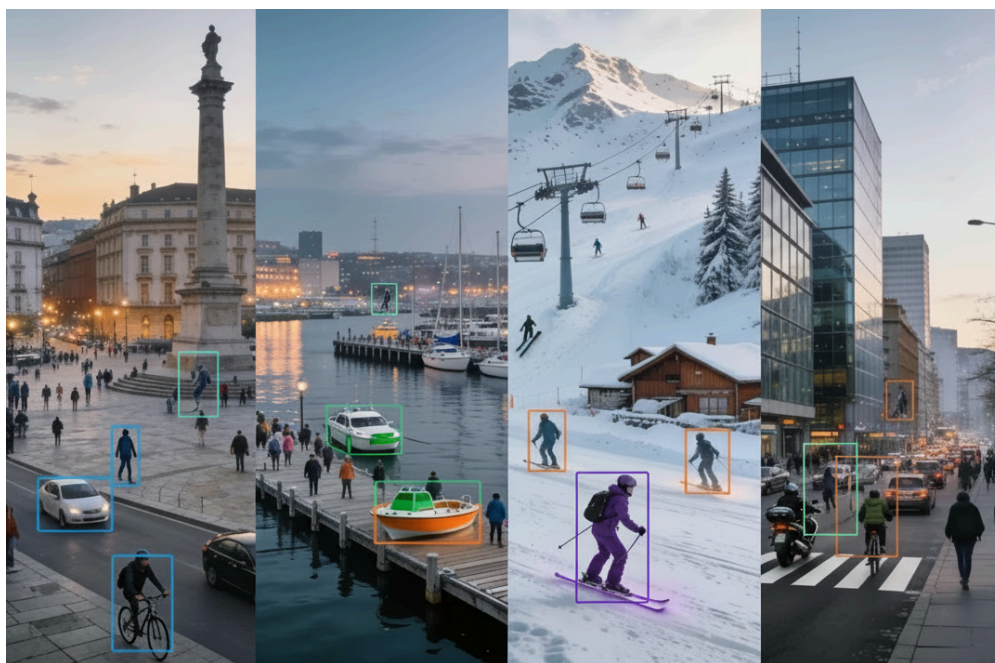


Place Publique

Observer la fréquentation d'espaces publics grâce aux flux video.



"On peut observer beaucoup de choses simplement en regardant." — Yogi Berra

Introduction

L'analyse des flux de personnes et de véhicules dans les espaces publics est devenue un enjeu stratégique majeur pour de nombreux acteurs territoriaux. Selon [une étude de MarketsandMarkets](#), le marché mondial de l'analyse vidéo intelligente (Video Analytics Market) devrait passer de 8,3 milliards de dollars en 2023 à 22,6 milliards de dollars d'ici 2028, avec un taux de croissance annuel composé de 22,3%. Cette croissance s'explique notamment par la



demande croissante en solutions de comptage et d'analyse statistique de fréquentation.

Les collectivités territoriales, gestionnaires de sites à forte affluence,, offices de tourisme, exploitants de stations de skis, de ports de plaisance et responsables de monuments historiques recherchent des outils permettant de :

- **Quantifier précisément les flux de visiteurs** pour
 - **Dimensionner les services publics**
 - Optimiser la **gestion des infrastructures** (parkings, transports, accès)
 - Améliorer la sécurité et la **gestion des foules** lors d'événements
- **Objectiver les décisions d'aménagement urbain** par des données factuelles
- **Mesurer l'impact de politiques publiques** (piétonnisation, mobilité douce, etc.)

Les villes européennes investissent massivement dans les systèmes intelligents de gestion du trafic. Selon [Juniper Research](#), ces solutions devraient générer 277 milliards de dollars d'économies pour les villes avant 2026, principalement grâce à la réduction des congestions et des temps de trajet (réduction de 33 heures par an et par conducteur en moyenne). En Chine, [une étude sur 100 villes congestionnées](#) a démontré que les systèmes



adaptatifs de signalisation réduisent les temps de trajet de 11% aux heures de pointe, tout en diminuant significativement les émissions de carbone.

Par ailleurs, la mesure précise des flux de visiteurs est devenue une priorité stratégique pour les gestionnaires d'espaces publics et touristiques en Europe. [Une étude collaborative](#) menée dans 518 sites naturels européens (avec plus de 1 267 observations de comptage) a démontré que le nombre total de visiteurs annuels constitue la variable déterminante pour évaluer la valeur économique récréative d'un site et justifier l'allocation de ressources. Cette importance est confirmée par les pratiques de terrain : en France, le [guide officiel de gestion des flux touristiques](#) recommande désormais le déploiement de systèmes de comptage automatisés comme outil essentiel d'aide à la décision pour les gestionnaires de sites.

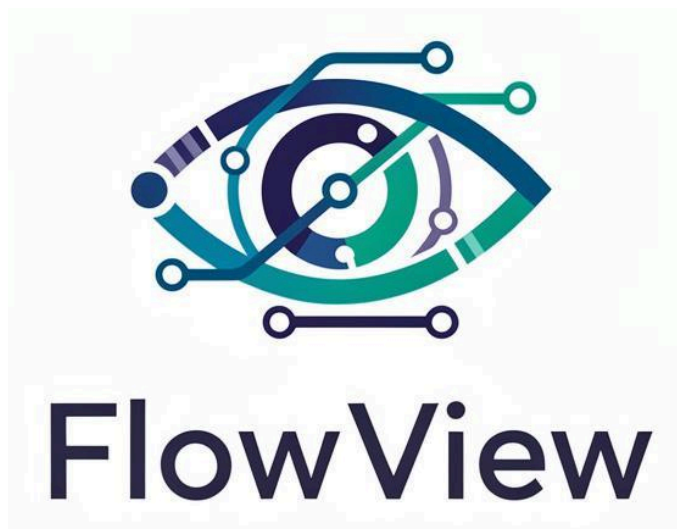
Les technologies de vision par ordinateur, en particulier les modèles de détection d'objets basés sur l'apprentissage profond (deep learning), offrent aujourd'hui des performances permettant d'automatiser ces analyses à partir de flux vidéo existants. Contrairement aux solutions traditionnelles nécessitant des capteurs spécialisés coûteux, l'approche par vision artificielle permet de valoriser des webcams déjà déployées, réduisant significativement les coûts d'infrastructure.

C'est dans ce contexte que s'inscrit le projet "Place Publique" : développer une solution complète, industrialisable et éthique de comptage automatisé d'objets animés (personnes, véhicules) à partir de flux vidéo, intégrant



l'ensemble de la chaîne de valeur depuis l'entraînement des modèles jusqu'à la production de statistiques exploitables.

Mise en situation professionnelle



Une entreprise que vous auriez pu créer

Vous faites partie d'une équipe de deux ou trois ingénieurs Data/IA spécialisés en vision par ordinateur, travaillant pour **FlowView**, une startup innovante spécialisée dans l'analyse vidéo intelligente. L'entreprise, créée il y a 3 ans, a déjà réalisé plusieurs PoC (Proof of Concept) pour des clients pilotes, mais souhaite désormais passer à l'échelle en mettant en commun les éléments génériques de son produit **Place-Publique**.

Généraliser et capitaliser les savoir-faire



Portefeuille de clients

FlowView a identifié plusieurs clients potentiels qui ont exprimé un besoin concret assez similaire :

1. Les commerçants de la [place de la comédie à Montpellier](#), (voir aussi [ce site](#) pour d'autres exemples) de même que les restaurants et cafés de la [promenade des Anglais à Nice](#) s'intéressent aux statistiques de fréquentation des espaces publics pour anticiper les besoins de renfort en employés ou en réapprovisionnement.
2. La [Ville de Béthune](#) cherche à quantifier les flux de véhicules (voitures, motos, vélos, trottinettes, bus, camions) et de piétons sur plusieurs axes stratégiques pour évaluer l'impact de la piétonnisation, anticiper le congestionnement urbain, et ajuster les plans de circulation.
3. La [Station de ski Les Arcs \(Savoie\)](#) : souhaite compter les piétons sur la place de la ville, et les skieurs dans les files d'attente des remontées mécaniques pour optimiser l'ouverture des télésièges en fonction de l'affluence et communiquer l'information en temps réel aux clients. La ville de [Brides les Bains \(Les 3 Vallées\)](#) a aussi exprimé un besoin identique.
4. Office de Tourisme de Reims : veut mesurer la fréquentation [autour de la cathédrale](#) pour mieux dimensionner l'accueil touristique et adapter les horaires d'ouverture des sites.
5. La [Marina du port de Lorient](#) et le [port de plaisance de Cap d'Ail](#): souhaitent automatiser le comptage des bateaux (voiliers, yachts, bateaux à moteur) entrant et sortant du port pour optimiser la gestion des places d'amarrage et produire des statistiques de fréquentation.



Expression du besoin

Chaque client dispose d'une ou plusieurs webcams publiques (flux vidéo en continu) déjà installées sur site. Ils souhaitent :

- **Identifier** et **compter** automatiquement entre 2 et 10 classes d'objets **selon leur contexte** (personnes, types de véhicules, types de bateaux, etc.)
- Disposer d'un **site web de centralisation du suivi des webcams**.
- Visualiser des **statistiques de fréquentation** (horaires, quotidiennes, hebdomadaires, mensuelles) sous forme de graphiques et tableaux de bord
- Recevoir des **alertes en cas d'anomalie** (forte affluence, objet inhabituel/inconnu détecté)
- Si possible, bénéficier d'un **système évolutif**, capable de s'adapter à l'apparition de nouveaux types d'objets

Le système doit **analyser les flux vidéo** à **périodicité régulière** (par exemple toutes les 5 minutes), être **robuste en cas de panne ou d'arrêt temporaire** d'une caméra, **compter les objets** détectés par classe, et **stocker ces décomptes** dans une base de données pour permettre l'analyse temporelle.

Votre mission

En tant qu'équipe technique, vous devez concevoir et développer une solution complète qui répond aux besoins de ces différents clients, tout en restant



générique et adaptable. Votre solution doit couvrir l'ensemble de la chaîne de valeur, avec dataOps, MLOps, devOps, suivi/maintenance, conformité:

1. Veille technique

Réalisez une veille technique sur les questions suivantes, et rédiger votre compte-rendu de veille dans un fichier (illustré et référencé) **veille.md**:

Choix des modèles et performances

1. Quels sont aujourd'hui les **principaux modèles de détection d'objets utilisés en vidéosurveillance** (YOLOv8/YOLO11, Faster R-CNN, DETR, etc.) et **quels compromis** font-ils entre vitesse d'inférence, précision (mAP) et consommation de ressources pour un déploiement temps réel sur du matériel limité ?
2. Quelles bonnes pratiques permettent d'**évaluer rigoureusement un modèle de détection d'objets** sur des scènes de caméras fixes (métriques mAP, précision/rappel par classe, IoU, analyse d'erreurs) afin de garantir la fiabilité des statistiques de comptage dans des conditions réelles (jour/nuit, pluie, foule dense) ?

Données, annotation et robustesse

3. Comment **sélectionner, combiner et annoter** des datasets (Roboflow Universe, datasets de trafic, webcams publiques) pour couvrir la **variété des contextes clients** (places, ports, stations de ski) tout en



contrôlant la taille du dataset pour un entraînement faisable sur laptop/Colab ?

4. Quelles **stratégies d'augmentation de données et d'adaptation de domaine** (changement de météo, saison, angle de caméra, résolution) sont pertinentes pour améliorer la robustesse d'un modèle de détection appliqué à des flux webcams hétérogènes et évolutifs ?

Architecture logicielle, DataOps et MLOps

5. Quelles **architectures applicatives** sont les plus adaptées pour relier flux vidéo, service d'inférence, base de données temporelle et frontend (Flask) : traitement batch périodique vs quasi temps réel, microservices, file de messages, et **quels impacts** sur la scalabilité et la tolérance aux pannes ?
6. Quels **outils et patterns MLOps** (MLflow, CI/CD, versioning de modèles, tests automatisés) sont recommandés pour assurer un cycle de vie maîtrisé des modèles de détection d'objets, depuis l'expérimentation jusqu'au déploiement reproductible en production (Docker, cloud/on premise) ?



Monitoring, dérive et réentraînement

7. Quelles **métriques** et quels **signaux** (statistiques de distribution, taux de détection, feedback utilisateur, anomalies sur les séries temporelles de comptage) permettent de **détecter efficacement la dérive de données ou de concepts** dans un système de vidéosurveillance algorithmique, et comment déclencher un processus de réentraînement pertinent ?
8. Quelles approches existent pour intégrer le **monitoring de modèles** (MLflow, dashboards, alertes) dans une application de vision par ordinateur en production, afin de suivre à la fois la **qualité des prédictions** et la **santé opérationnelle** du pipeline de traitement vidéo ?

RGPD, AI Act et éthique

9. Comment les cadres réglementaires RGPD et AI Act classifient-ils les systèmes de vidéosurveillance algorithmique, quelles obligations en découlent (base légale, DPIA, registres, transparence) et quelles pratiques techniques (anonymisation, absence de stockage vidéo, agrégation) sont recommandées pour rester conforme ?
10. Quels sont les **principaux enjeux éthiques liés au comptage automatisé de personnes et de véhicules** dans l'espace public (risque de surveillance de masse, biais sur certaines populations, réutilisation des données) et quelles **mesures de conception** (privacy by design,



limitation des finalités, gouvernance des modèles) peuvent en limiter les risques ?

Pour aller plus loin : Robustesse, dérive et classes inconnues

11. Comment passer d'un **détecteur « fermé »** classique (qui choisit toujours une classe connue) à une **approche d'open-set object detection** capable de signaler des objets inconnus ou « autre » dans une scène réelle de vidéosurveillance ?
12. Quelles stratégies (seuils de confiance, incertitude, bruit de fond, pseudo-labels, zones d'attention) permettent de **distinguer un vrai « objet inconnu »** d'un simple faux positif ou d'un bruit de fond dans un modèle de détection ?
13. Comment **exploiter les objets détectés comme « inconnus/autre » pour déclencher un processus de ré-annotation / ré-entraînement**, et transformer progressivement ces inconnus en nouvelles classes utiles pour le client (nouveaux types de véhicules, engins de chantier, équipements saisonniers, etc.) ?
14. Quelles **métriques ou protocoles d'évaluation** spécifiques à l'open-set (taux de détection d'inconnus, calibration de la confiance, analyse des erreurs sur les classes « autre ») peut-on utiliser pour juger la robustesse réelle du système dans un environnement ouvert ?

Pour aller plus loin : Backend production : de Flask à gUnicorn

15. Pourquoi le serveur intégré de Flask n'est-il pas une **solution adaptée à la production** (concurrence limitée, pas de gestion robuste des



workers, absence de fonctionnalités avancées de log et de monitoring)
et en quoi un serveur WSGI comme gUnicorn répond-il à ces limites?

16. Comment **architecturer un déploiement typique Flask + gUnicorn** (+ éventuellement Nginx) : rôle de chaque composant, gestion des workers, équilibrage de charge, gestion des timeouts pour les requêtes d'inférence lourdes (vision par ordinateur)?
17. Quelles **options de configuration gUnicorn** (nombre de workers, worker class sync / gevent, limites de temps, logs) sont particulièrement critiques pour une API de détection d'objets qui doit traiter simultanément plusieurs flux webcams?
18. Comment **intégrer gUnicorn dans la chaîne CI/CD existante** du projet (Dockerfile, commande d'entrée, health checks) afin de passer facilement d'un serveur de dev Flask à un service de prod reproductible?

Pour aller plus loin : Déploiement cloud et scalabilité

19. Quelles sont les **principales options de déploiement d'un service de vision par ordinateur dans le cloud** (VM classique, conteneur Docker sur Kubernetes, serverless type Lambda/Cloud Functions, endpoints managés type Vertex AI / SageMaker) et quels sont leurs avantages / limites pour un traitement de flux vidéos périodiques ou temps quasi réel ?
20. Comment **concevoir une architecture scalable** où le service d'inférence est conteneurisé (Docker) et déployé sur un orchestrateur (Kubernetes, ECS, AKS, GKE) afin d'ajuster automatiquement le nombre



de workers de détection en fonction du nombre de webcams et du volume de requêtes ?

21. Dans quels cas un **déploiement « edge »** (Docker sur une machine sur site, cluster on-premise) est-il préférable au full cloud pour ce type d'application (latence, confidentialité, coûts de bande passante) et comment intégrer cette contrainte dans l'architecture proposée pour ce projet?
22. Quelles **bonnes pratiques de monitoring et d'observabilité** cloud (logs centralisés, métriques, traces, dashboards) doit-on ajouter à MLflow pour suivre à la fois la santé du cluster (CPU/GPU, mémoire) et la qualité des prédictions (drift, erreurs, temps de réponse) à grande échelle ?

2. Entraînement et spécialisation de modèles de détection d'objets

- Choisir une architecture de **modèle de détection d'objets** adaptée (YOLO, etc.)
- **Sélectionner et préparer** des datasets appropriés
- **Entraîner ou fine-tuner** le modèle sur les classes d'intérêt
 - Évaluer rigoureusement les **performances** (métriques mean average precision, précision et rappel par classe)
 - [optionnel] Spécifier et développer une **approche de détection d'objets à classes ouvertes**
- Tracker les **expériences** et **versionner** les modèles avec un outil MLOps

3. Architecture applicative et déploiement



- **Concevoir l'architecture globale du système** (frontend, backend, base de données, service d'inférence)
- **Développer un frontend web** en Flask permettant l'exploitation d'un modèle entraîné pour un client donné:
 - L'enregistrement de webcams (URL, nom, localisation)
 - La sélection des classes d'objets à suivre
 - La visualisation de dashboards avec statistiques temporelles
- Implémenter un **service d'inférence** qui :
 - Lit périodiquement les flux vidéo abonnés (ou une image jpg/png publiée sur le site d'exploitation des webcams)
 - Communique avec le modèle via API, et applique le modèle de détection sur les images extraites périodiquement
 - Produit
 - l'annotation des classes d'objets présentes dans l'image
 - retourne l'image annotée (pour visualisation)
 - retourne le compte des objets par classe
 - Stocke les résultats en base (sqlite ou CSV)
- Mettre en place un **pipeline CI/CD avec tests unitaires pour le déploiement automatisé**
 - [optionnel] A défaut de disposer de ressources Cloud ou On Premise, si la mise en production n'est faite que sur PC portable personnel, la mise en production automatisée ne sera pas exigée.
- [optionnel] Monter en compétence en faisant une **mise en production du modèle et de l'application avec gUnicorn**



- Proposer une **architecture de production scalable** (dans un fichier architecture.md)

4. Monitoring et gestion de la dérive

- Définir des **métriques de qualité du modèle en production**
- Définir une ou plusieurs approches pour détecter la **dérive du modèle (data drift, concept drift)**
- Identifier les **situations nécessitant un ré-entraînement** (nouveaux objets, changement de comportement)
- Documenter ces éléments ainsi que le **processus de mise à jour du modèle** (architecture.md)
- [Optionnel] Développer les métriques de détection de la dérive, et les ajouter à votre projet
- [Optionnel] Intégrer MLflow pour le suivi de la performance en production et l'envoi d'alertes de dérive

5. Conformité réglementaire (RGPD, AI Act)

- **Analyser les risques** liés à la vidéosurveillance algorithmique
- Évaluer la **conformité au RGPD** (base légale, minimisation, DPIA)
- Positionner le système au regard de l'**AI Act européen**
- Proposer des **mesures techniques de mise en conformité**
- Rédiger une **note de conformité** destinée aux clients (conformite.md)

Contraintes techniques



- Le développement doit être réalisé en Python
- Le code doit être versioné sur Git (GitHub)
- Le frontend sera développé avec Flask (+ par exemple templates Jinja, Chart.js pour les graphiques)
- Les modèles doivent être entraînaables sur laptops d'étudiants ou Google Colab gratuit
- Le déploiement sera simulé en local, mais l'architecture proposée doit être transposable en production (Docker, cloud / on premise)
- Utilisation d'outils MLOps standards (MLflow recommandé)
- Les datasets doivent être open source et libres de droits

Difficultés envisagées et recommandations

- Ne pas chercher à entraîner un modèle à partir de zéro. Utiliser des modèles préentraînés, et éventuellement les fine-tuner en fin de projet sur un petit nombre d'images issues de webcams (à annoter manuellement) , ou sur un petit nombre d'images de datasets publics..
- Choisir de traiter un maximum de 2 à 3 webcams du même type (ayant les mêmes classes à identifier, par ex piétons/véhicules, ou bateaux), mais pas des webcams avec des caractéristiques trop différentes (par ex pistes de ski + port de plaisance)
- Choisir des webcams ayant des flux exploitables par OpenCV, ou à défaut ayant des sites web qui mettent à jour régulièrement une image (jpg ou png). L'utilisation de webcams dont le client serait propriétaire n'est possible que dans un projet réel en entreprise. À défaut d'accord



des propriétaires des webcams publiques, l'exploitation des flux video bruts peut être sujette à des restrictions légales.

- Se souvenir que l'objectif du projet est le dénombrement (à gros grain) d'objets à des fins statistiques, pas la détection parfaite d'objets.
- Laisser de côté les demandes facultatives/optionnelles, et finir le projet avant de tenter d'y répondre.
- Ne pas développer complètement les architectures que vous allez proposer pour la production, mais envisager les architectures cloud uniquement de manière prospective
- Dans le contexte d'un déploiement sur laptop, les demandes de CI/CD sont à alléger.
- L'analyse de dérive de modèle n'est à spécifier que sur le papier (veille et architecture), mais à développer uniquement si il vous reste du temps.
- De même, ne pas chercher à développer les demandes de Monitoring MLOps en production (juste développer le monitoring des expériences d'entraînement de modèles, et uniquement en cas de fine-tuning)
- Organisez bien votre équipe pour vous répartir les tâches, et imposer très tôt une structure GIT standard.



- Un dépôt Git complet et documenté, nommé place_publicue
- Un rapport de veille, constitué des réponses aux questions de veille, avec des liens et illustrations (fichier veille.md)
- Des notebooks jupyter ou colab d'entraînement et d'évaluation
- Une application web Flask fonctionnelle (même en local)
- optionnellement une version industrialisable de l'application web (passage de Flask à gUnicorn)
- Une démonstration du système en action
- Une documentation technique détaillée (fichier readme.md)
- Un rapport de conformité réglementaire (fichier conformite.md)
- Une proposition d'architecture de production (architecture.md), contenant optionnellement les processus de gestion de dérive du modèle et le monitoring en production
- Une présentation orale (avec un document powerpoint ou équivalent), devant un panel de clients et d'investisseurs





Rendu

L'ensemble des livrables mentionnés ci-dessus sera fourni dans le **dépôt GIT place_public** de l'équipe.

La **présentation de 20 minutes**, suivie d'une **démo de 10 minutes**, et un échange de **questions/réponses**, permettra d'évaluer l'acquisition des compétences nécessaires à la réalisation du projet.

Compétences visées

Ce projet permet de développer et d'évaluer les compétences suivantes, en correspondance avec les blocs de compétences du Master 2 Data/IA (RNCP40573: Expert en informatique et systèmes d'information) :

RNCP40573BC01 : Définir une stratégie de systèmes d'information

Ce bloc valide votre capacité à analyser l'état de l'art et le cadre légal pour orienter les choix technologiques de l'entreprise.

- **Veille technologique et réglementaire** : Réaliser une veille structurée sur les modèles de détection (YOLOv8/11 vs Faster R-CNN) et sur l'évolution des performances en temps réel.
- **Analyse de conformité et éthique** : Évaluer les enjeux juridiques liés à la vidéosurveillance algorithmique (RGPD, AI Act européen) et proposer



des mesures de "Privacy by Design" (anonymisation, limitation des finalités).

- **Préconisation de solutions** : Recommander des solutions innovantes et écoresponsables, comme la valorisation de webcams existantes pour réduire les coûts d'infrastructure.

RNCP40573BC02 : Piloter des projets informatiques

Ce bloc évalue votre aptitude à transformer un besoin client complexe en une solution opérationnelle.

- **Cadrage et étude d'opportunité** : Analyser les problématiques de clients variés (commerçants, stations de ski, marinas) pour définir une solution générique de comptage automatisé.
- **Communication et documentation** : Produire des livrables professionnels (rapport de veille, note de conformité, documentation technique) et présenter oralement les résultats devant un jury.
- **Travail en équipe** : Coordonner les tâches au sein de l'équipe FlowView et gérer un dépôt Git standardisé pour assurer la capitalisation des savoir-faire.

RNCP40573BC03 : Concevoir et développer une application informatique

Ce bloc valide la maîtrise du cycle de développement logiciel de l'application Place Publique.

- **Architecture applicative** : Concevoir une architecture reliant les flux vidéo, le service d'inférence, la base de données temporelle et le frontend Flask.



- **Développement d'applications métiers** : Implémenter un tableau de bord permettant l'enregistrement de webcams et la visualisation de statistiques de fréquentation via des graphiques et tableaux de bord.
- **Suivi de la qualité** : Rédiger et exécuter des tests unitaires pour garantir le bon fonctionnement du système de comptage par classe.

RNCP40573BC05 (Option) : Construire et implémenter des modèles de big data et d'IA

C'est le cœur technique de votre spécialisation en Vision par Ordinateur.

- **Conception de modèles d'apprentissage** : Sélectionner l'architecture YOLO adaptée au besoin métier et réaliser le fine-tuning ou le transfer learning sur des datasets spécialisés (trafic, maritime, ski).
- **Évaluation et optimisation** : Analyser rigoureusement les performances des modèles à l'aide des métriques mAP (mean Average Precision), précision et rappel par classe.
- **Ingénierie des données** : Préparer, annoter et enrichir les datasets via des stratégies d'augmentation de données pour assurer la robustesse face aux variations climatiques ou lumineuses.
- **Développement de programmes d'IA** : Implémenter l'API d'inférence capable d'analyser périodiquement les images et de produire des métadonnées de comptage.

RNCP40573BC06 (Option) : Concevoir et déployer des infrastructures DevOps automatisées

Ce bloc valide l'industrialisation et la mise en production de la solution IA.



- **Automatisation du déploiement (CI/CD) :** Mettre en place des pipelines automatisés et proposer une conteneurisation via Docker pour assurer la portabilité de l'application.
- **Monitoring et observabilité :** Intégrer des outils comme MLflow pour le suivi des expériences et la détection de la dérive des modèles (data/concept drift) en production.
- **Scalabilité et performance :** Optimiser l'architecture pour le passage à l'échelle (utilisation de gUnicorn, réflexion sur le déploiement Cloud ou Edge) afin de traiter simultanément plusieurs flux vidéo.

Base de connaissances

Cette section regroupe des ressources pédagogiques essentielles pour la compréhension et la réalisation du projet. Les ressources sont organisées par thématique.

1. Vision par ordinateur et modèles YOLO

1.1 Introduction à YOLO et détection d'objets

- Tutoriels complets Ultralytics YOLO : <https://docs.ultralytics.com/guides/>
 - Guide officiel couvrant la formation, le déploiement et l'optimisation des modèles YOLO (YOLOv8, YOLO11)



- YOLO Algorithm for Object Detection Explained :
<https://www.v7labs.com/blog/yolo-object-detection> - Explication détaillée de l'architecture YOLO et de son fonctionnement
- Comprendre les étapes clés d'un projet de vision par ordinateur :
<https://docs.ultralytics.com/fr/guides/steps-of-a-cv-project/> - Guide pratique sur la structuration d'un projet de computer vision
- Object Detection with YOLO: Hands-on Tutorial :
<https://neptune.ai/blog/object-detection-with-yolo-hands-on-tutorial>
- Tutoriel pratique avec implémentation en TensorFlow/Keras

1.2 Fine-tuning et Transfer Learning

- How to Use Fine Tune YOLOv8 :
<https://yolov8.org/how-to-use-fine-tune-yolov8/> - Guide complet sur le fine-tuning de YOLOv8 sur des datasets personnalisés
- Fine-tune YOLOv8 models for custom use cases :
<https://docs.voxel51.com/tutorials/yolov8.html> - Tutoriel avancé sur l'évaluation et le fine-tuning avec FiftyOne

1.3 Tutoriels vidéo pratiques

- How to Train YOLO Object Detection Models in Google Colab :
<https://www.youtube.com/watch?v=r0RspiLG260> - Tutoriel complet (entraînement YOLO11/YOLOv8 sur GPU gratuit)



- YOLOv11: How to Train for Object Detection on a Custom Dataset : <https://www.youtube.com/watch?v=etjkjZoG2F0> - Guide pas à pas (1h11) couvrant datasets, annotation, entraînement et déploiement

2. Accès aux flux webcam et traitement vidéo

2.1 OpenCV et capture vidéo

- Getting Started with Videos (OpenCV Documentation) : https://docs.opencv.org/4.x/dd/d43/tutorial_py_video_display.html - Documentation officielle sur la capture et l'affichage de vidéos
- Display the webcam in Python using OpenCV : <https://gist.github.com/tedmiston/6060034> - Code simple pour afficher le flux webcam

2.2 Streaming vidéo et WebSocket

- OpenCV - Stream video to web browser/HTML page : <https://pyimagesearch.com/2019/09/02/opencv-stream-video-to-web-browser-html-page/> - Tutoriel pour streamer une webcam vers un navigateur avec Flask
- Streaming Live Video from Webcam to Web Using Python : <https://github.com/joshting/python-cv2-webcam-stream> - Exemple de code utilisant WebSocket pour diffuser un flux vidéo en temps réel



3. MLOps et suivi d'expériences avec MLflow

3.1 Documentation et quickstart MLflow

- MLflow Tracking : <https://mlflow.org/docs/latest/ml/tracking/> - Documentation officielle sur le suivi des paramètres, métriques et modèles
- MLflow Tracking Quickstart : <https://mlflow.org/docs/latest/ml/tracking/quickstart/> - Guide de démarrage rapide avec exemples de code
- MLflow Tutorials and Examples : <https://mlflow.org/docs/latest/ml/tutorials-and-examples/> - Collection de tutoriels pour différents cas d'usage

3.2 Monitoring de modèles

- MLflow Model Monitoring in AI Studio : <https://community.datascience.hp.com/hackathons-91/mlflow-model-monitoring-in-ai-studio-350> - Tutoriel sur le monitoring et le versioning de modèles
- MLflow Python Tutorial - ML Model Experiment Tracking : <https://www.youtube.com/watch?v=kn5lWgTTjCw> - Tutoriel vidéo sur le tracking d'expériences



4. Conformité réglementaire : RGPD et IA Act

4.1 RGPD et vidéosurveillance

- CNIL Vidéosurveillance : RGPD et légalité : <https://veesion.io/cnil-videosurveillance-rgpd-et-legalite/> - Guide sur les obligations RGPD pour la vidéosurveillance
- Vidéosurveillance & IA : conformité RGPD et libertés : <https://www.adequacy.app/blog/videosurveillance-ia-conformite-rgpd> - Article détaillé sur la vidéosurveillance algorithmique et la conformité
- RGPD et caméras intelligentes (EY Avocats) : https://www.avocats.ey.com/fr_fr/droit-des-affaires/rgpd-et-cameras-intelligentes - Position juridique sur les caméras "intelligentes" et l'AIPD
- Vidéosurveillance : la CNIL sanctionne et se positionne sur l'IA : <https://www.ghr.fr/europe-numerique/actualites/videosurveillance-la-cnil-sanctionne-et-se-positionne-sur-le-recours-a-l-ia> - Analyse des décisions récentes de la CNIL

4.2 IA Act européen

- The EU AI Act: Regulatory standard for responsible AI : <https://www.intenseye.com/blog/the-eu-ai-act-regulatory-standard-for-responsible-ai-blog> - Vue d'ensemble du cadre réglementaire européen



- Video surveillance technology and the new EU AI Act : <https://www.dallmeier.com/about-us/dallmeier-blog/video-security-technology-and-biometric-facial-recognition-under-new-eu-ai-act> - Impact de l'IA Act sur la vidéosurveillance et la reconnaissance faciale
- Regulating computer vision & the ongoing relevance of AI ethics : <https://montreal.ethics.ai/regulating-computer-vision-the-ongoing-relevance-of-ai-ethics/> - Analyse critique de la régulation de la vision par ordinateur
- EU AI Act Compliance Checker : <https://artificialintelligenceact.eu/assessment/eu-ai-act-compliance-checker/> - Outil d'évaluation de conformité à l'IA Act

5. Datasets recommandés

Pour ce projet, privilégiez des datasets annotés disponibles sur :

- Roboflow Universe : <https://universe.roboflow.com/object-detection> - Collections de datasets pour la détection d'objets (trafic, surveillance, etc.)
- Google Dataset Search : <https://datasetsearch.research.google.com/> - Moteur de recherche de datasets académiques
- UCI Machine Learning Repository : <https://archive.ics.uci.edu/> - Repository de datasets pour le machine learning